



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Faber Henrique Zacarias Xavier

TUTORIAL PRÁTICO: CONSTRUÇÃO DE ESTEIRA CI/CD

1. Objetivo e Metodologia

Esta atividade acadêmica visa à construção incremental de um workflow de CI/CD utilizando GitHub Actions. O foco não é apenas a automação, mas a compreensão do papel de cada bloco lógico no ciclo de vida de desenvolvimento.

Como usar este material:

- O script Python e os testes já estão prontos; o foco é a esteira.
- O tutorial não fornece o arquivo YAML final: ele orienta as decisões técnicas que resultam no workflow.
- Cada etapa possui um validador obrigatório para confirmar a configuração correta antes de prosseguir.

2. Projeto Base

O aluno receberá um repositório com a seguinte estrutura:

```
ci-cd-lab/  
├── app/ pipeline.py  
├── data/ sales.csv  
├── tests/ test_pipeline.py  
├── requirements.txt  
└── .github/ workflows/
```

Validador Inicial: Execute localmente o script e os testes (pytest). Se o projeto não rodar localmente, a esteira não terá uma base confiável.

Etapa I: Definindo o Gatilho (Trigger)

A primeira decisão técnica é definir quando a esteira deve disparar. No GitHub Actions, usamos o comando 'on' para configurar gatilhos.

Instrução: Configure o workflow para ser ativado exclusivamente em eventos de 'push' na branch 'feature/esteira'. Isso permite validar a configuração sem impactar a branch principal (main).

Validador: Realize um push. Uma execução deve aparecer automaticamente na aba 'Actions' do GitHub.

Etapa II: O Job de Validação

Jobs representam agrupamentos lógicos de ações. Com o comando 'runs-on', define-se o ambiente de execução (runner).

Instrução: Crie um job chamado 'validate' utilizando uma máquina virtual 'ubuntu-latest'.

Validador: A execução deve iniciar no GitHub e exibir o job 'validate', mesmo que ainda não possua etapas úteis.

Etapa III: Checkout e Runtime

Para que a esteira processe o código, ela precisa 'baixar' o conteúdo do repositório (Checkout) e preparar o interpretador Python.

Conceito: A action 'actions/checkout' torna o código visível para o runner, enquanto 'actions/setup-python' configura a versão específica (3.11).

Validador: Verifique nos logs se as etapas de 'Checkout' e 'Configurar Python' foram concluídas com sucesso.

Etapa IV: Dependências e Qualidade (Linting)

Antes dos testes, é fundamental validar o estilo e a sintaxe. O 'flake8' é utilizado para garantir que o código atenda aos padrões definidos.

Instrução: Adicione etapas para instalar as dependências (pip install -r requirements.txt) e executar o linting nas pastas 'app' e 'tests'.

Validador: O workflow deve exibir a etapa 'Lint'. Se falhar, corrija o código-fonte até que a validação passe.

Etapa V: Testes Automatizados e Régua de Qualidade

A Integração Contínua (CI) exige a validação de comportamento. O 'pytest' confirma que o código continua funcionando conforme o esperado.

Desafio Incremental: Para aumentar a 'régua de qualidade', integre ferramentas adicionais como 'mypy' (tipagem), 'bandit' (segurança) ou 'radon' (complexidade).

Validador: O workflow deve finalizar com sucesso passando por Lint e Testes.

Etapa VI: Geração e Entrega de Artefatos

Nesta fase, a esteira deixa de apenas validar e passa a gerar resultados. O 'artifact' é o objeto que conecta a Integração Contínua (CI) à Entrega Contínua (CD).

Instrução: Execute o script Python para gerar o arquivo 'output/summary.csv' e utilize uma step de upload para publicá-lo como artefato do workflow.

Validador: Ao final da execução, deve existir um artefato disponível para download na página do workflow.

Etapa VII: Promoção e Entrega Contínua (CD)

O CD (Continuous Delivery) envolve promover o artefato aprovado para o ambiente final. O deploy deve ocorrer apenas após o sucesso da validação e em condições controladas.

Instrução: Crie um segundo job chamado 'deploy', dependente do job 'validate'. Configure uma condição para que ele execute apenas quando o evento ocorrer na branch 'main'.

Validador: Em pushes para a 'feature', apenas a validação roda. Após o merge na 'main', o job de deploy deve ser ativado.

4. Reflexão Técnica

1. O que, nesta atividade, pertence ao CI e o que pertence ao CD?
2. Qual a diferença conceitual entre o script Python e a esteira que o valida?
3. Se houvesse um ambiente de staging, em que ponto ele entraria no fluxo?

5. Guia de Apoio Técnico

Dicas e Erros Comuns (Troubleshooting)

- Indentação YAML: O erro mais comum é o desalinhamento de espaços. Use editores que suportam YAML para facilitar.
- Case Sensitive: Nomes de branches e chaves de configuração diferenciam maiúsculas de minúsculas.
- Segredos e Variáveis: Nunca deixe senhas expostas; use o 'GitHub Secrets' se necessário.

Glossário Técnico

- Runner: Máquina virtual hospedada onde a esteira é executada.
- Artifact: Objeto digital (arquivo) gerado e preservado pela esteira.
- Linting: Análise estática para encontrar erros de estilo ou sintaxe.